

Technical Report: Machine Learning Classification and Regression Pipeline

Abstract

This report provides a technical analysis of a Python implementation that applies classical machine-learning methods to student performance data. The program implements linear regression, several binary classification models, logistic regression decision-boundary analysis, and a manual matrix-based classifier formulation. The analysis covers the computational workflow, mathematical foundations, implementation details, and reproducibility considerations.

1 Introduction

The project implements a classical machine-learning workflow in Python using Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn. The input is an Excel dataset containing student-related features. The implementation models both continuous outcomes and binary pass/fail decisions using linear regression and classification algorithms.

2 Project Overview

The computational pipeline consists of:

1. Loading the Excel input file.
2. Preparing feature matrices by removing non-predictive columns.
3. Training a linear regression model for continuous prediction.
4. Training Ridge, SGD, and Logistic Regression classifiers.
5. Computing evaluation measures such as accuracy, precision, and confusion matrices.
6. Analysing a two-feature logistic regression decision boundary.
7. Reconstructing a classifier using explicit matrix operations.
8. Packaging generated artifacts for reproducibility.

3 Data Processing

The implementation reads the Excel file into a Pandas dataframe. Columns representing the target value and metadata are removed before model fitting. For classification, the categorical pass label is converted to a binary representation:

$$y_i = \begin{cases} 0, & \text{if the student fails,} \\ 1, & \text{if the student passes.} \end{cases}$$

4 Linear Regression

The regression component learns a linear relationship:

$$\hat{y} = w^T x + b,$$

where x is the feature vector, w contains the learned coefficients, and b is the intercept. The implementation uses Scikit-learn's LinearRegression estimator and exports the learned parameters.

5 Classification Models

Three classifiers are trained.

5.1 Ridge Classifier

Ridge classification adds regularization to a linear decision function. A typical regularized objective can be written as:

$$\min_w ||Xw - y||_2^2 + \lambda ||w||_2^2.$$

5.2 Logistic Regression

Logistic regression models the probability of the positive class as:

$$P(y = 1|x) = \sigma(w^T x + b),$$

where:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

The decision boundary occurs when the model score is zero:

$$w^T x + b = 0.$$

5.3 SGD Classifier

The stochastic gradient descent classifier updates its parameters iteratively using sampled training examples. Its optimization behavior depends on the Scikit-learn configuration used by the estimator.

6 Evaluation Procedure

The implementation calculates accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

precision:

$$\text{Precision} = \frac{TP}{TP + FP},$$

and confusion matrices for the classifiers.

7 Decision Boundary Analysis

A logistic regression model is trained using only midterm and final exam scores. The decision boundary is derived from:

$$b + w_1x_1 + w_2x_2 = 0.$$

Therefore:

$$x_2 = -\frac{b}{w_2} - \frac{w_1}{w_2}x_1.$$

8 Manual Matrix Formulation

The project also demonstrates a manual linear classifier calculation. A bias term is inserted into the feature matrix:

$$X = [1, x_1, x_2].$$

The implementation computes matrix products involving:

$$(X^T X)^{-1} X^T$$

and derives classifier weights through explicit numerical operations.

9 Implementation Details

The program is adapted for Google Colab execution. It automatically installs dependencies, handles the Excel input file, creates output directories, stores numerical results, exports tables, and creates a compressed archive containing the generated artifacts.

10 Reproducibility Considerations

The implementation supports reproducibility through explicit input handling, consistent file organization, and export of learned parameters. However, no fixed random seeds or train/test separation are defined in the source code; therefore, the reported metrics reflect evaluation on the fitted dataset rather than performance on an independent test set.

11 Limitations

The implementation is educational and demonstrates several algorithmic concepts. Because the models are evaluated on the same data used for training, the resulting metrics should not be interpreted as estimates of generalization performance. Additional validation procedures would be required for scientific benchmarking.

12 Conclusion

This project demonstrates a coherent classical machine-learning workflow that combines regression, classification, visualization, mathematical reconstruction, and reproducible artifact generation. It provides a practical connection between mathematical formulations and executable machine-learning procedures.